



## University of Chittagong

Department of Computer Science and Engineering

---

# Machine Learning Assignment

Perceptron and Hidden Markov Model

Course Code: CSE 815

---

### Submitted To

Prof. Dr. Mohammad Osiur Rahman  
Department of Computer Science and  
Engineering  
University of Chittagong

### Submitted By

Md. Kais  
Student ID: 20701003  
Department of Computer Science and  
Engineering  
University of Chittagong

Submission Date: September 14, 2026

---

# Declaration and Scope

---

This Assignment was prepared for Prof. Dr. Mohammad Osiur Rahman at University of Chittagong from the current source code, datasets, tests, `README.md` files, and project reports in the workspace. It describes the actual implementation rather than an imagined library version. The fixed header shows CSE 815 on the left and 20701003 on the right on every page; every footer contains only the centered page number.

The applications are educational. The Perceptron student points are user-created teaching examples and the HMM coins form a probability puzzle; neither project uses personal records.

## Algorithms and problems covered

| Algorithm  | Problem title                                                                 | Architecture                       |
|------------|-------------------------------------------------------------------------------|------------------------------------|
| Perceptron | Predicting a synthetic student Pass/Fail label from study time and attendance | Flask API trains; browser animates |
| HMM        | Guessing which hidden coin generated a visible H/T sequence                   | Browser decodes locally            |

---

# Contents

---

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Declaration and Scope</b>                                            | <b>2</b>  |
| <b>1 Perceptron</b>                                                     | <b>7</b>  |
| 1.1 Problem Description . . . . .                                       | 7         |
| 1.2 Algorithm Description . . . . .                                     | 7         |
| 1.2.1 The chalk-line explanation . . . . .                              | 7         |
| 1.2.2 A numerical step . . . . .                                        | 8         |
| 1.2.3 Core code and data flow . . . . .                                 | 8         |
| 1.3 Dataset Description . . . . .                                       | 9         |
| 1.4 Training . . . . .                                                  | 10        |
| 1.4.1 How to run and train . . . . .                                    | 10        |
| 1.4.2 Parameters a learner may change . . . . .                         | 11        |
| 1.5 Website Design and Answer Flow . . . . .                            | 11        |
| 1.6 Comprehensive User Manual . . . . .                                 | 12        |
| 1.6.1 Screen guide . . . . .                                            | 12        |
| 1.6.2 Recorded website experiment: a learnable class boundary . . . . . | 13        |
| 1.6.3 Every input and control . . . . .                                 | 16        |
| 1.6.4 How to perform a successful experiment . . . . .                  | 17        |
| 1.6.5 How to read every output . . . . .                                | 18        |
| 1.6.6 Troubleshooting . . . . .                                         | 18        |
| 1.7 Conclusion . . . . .                                                | 18        |
| <b>2 Hidden Markov Model</b>                                            | <b>19</b> |
| 2.1 Problem Description . . . . .                                       | 19        |
| 2.2 The Algorithm . . . . .                                             | 19        |
| 2.2.1 Model pieces . . . . .                                            | 19        |

|                                           |                                                                                   |           |
|-------------------------------------------|-----------------------------------------------------------------------------------|-----------|
| 2.2.2                                     | Viterbi: the single best hidden route . . . . .                                   | 20        |
| 2.2.3                                     | Forward-backward: belief at each position . . . . .                               | 20        |
| 2.3                                       | Dataset Description . . . . .                                                     | 21        |
| 2.4                                       | Training and Parameter Experimentation . . . . .                                  | 22        |
| 2.5                                       | Website . . . . .                                                                 | 22        |
| 2.6                                       | Comprehensive User Manual . . . . .                                               | 22        |
| 2.6.1                                     | Recorded website experiment: seven heads with a strongly biased<br>coin . . . . . | 23        |
| 2.7                                       | Conclusion . . . . .                                                              | 27        |
| <b>Overall Conclusion</b>                 |                                                                                   | <b>28</b> |
| <b>A Verification and Reproduction</b>    |                                                                                   | <b>29</b> |
| A.1                                       | Commands verified in this workspace . . . . .                                     | 29        |
| A.2                                       | One-page run checklist . . . . .                                                  | 29        |
| <b>B Glossary for a First-Time Reader</b> |                                                                                   | <b>30</b> |
| <b>References</b>                         |                                                                                   | <b>31</b> |

---

# List of Figures

---

|     |                                                                                                                                                                                                                                   |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Perceptron website request and response flow. . . . .                                                                                                                                                                             | 12 |
| 1.2 | Stage 1 - initial Perceptron page. The classroom map is empty, the student counter is zero, $W = [0, 0]$ , $b = 0$ , and no decision boundary exists yet. . .                                                                     | 13 |
| 1.3 | Stage 2 - input prepared. The Clearly Separable preset adds ten labeled students: five red Fail examples and five green Pass examples. Training has not started, so the displayed weights and bias remain zero. . . . .           | 14 |
| 1.4 | Stage 3 - one sample visit. The dashed ring identifies the active training point, while the telemetry reports its target, prediction, error, updated weights, bias, accuracy, epoch, and step. . . . .                            | 15 |
| 1.5 | Stage 4 - final output. The page reports “Converged - 100% correct” after epoch 23 and step 230, with $W = [0, 7]$ , $b = -30$ , and the blue boundary $7p_2 - 30 = 0$ . . . . .                                                  | 16 |
| 2.1 | Hidden Coin Detective inference flow. . . . .                                                                                                                                                                                     | 22 |
| 2.2 | Stage 1 - initial HMM page. The default nine-toss example is already decoded, showing the complete interface: observations, probability controls, Viterbi route, timeline, posterior bars, and plain-language guide. . . . .      | 23 |
| 2.3 | Stage 2 - input prepared but not decoded. The observation field contains seven Heads; Biased-heads is 90%, Fair-stay is 70%, and Biased-stay is 85%. The old “Decoded 9 tosses” result remains visible until Decode is pressed. . | 24 |
| 2.4 | Stage 3 - HMM output. For seven Heads under the stated probabilities, Viterbi returns Biased $\times 7$ with an exact path-and-observations probability of 9.02%. . . . .                                                         | 25 |
| 2.5 | Stage 4 - additional posterior information. At toss 5, which is H, the forward-backward result assigns 9.9% to Fair and 90.1% to Biased. The two bars total 100%. . . . .                                                         | 26 |

---

## List of Tables

---

|     |                                               |    |
|-----|-----------------------------------------------|----|
| 1.1 | Perceptron implementation map. . . . .        | 9  |
| 1.2 | Perceptron dataset fields. . . . .            | 10 |
| 1.3 | Perceptron tuning guide. . . . .              | 11 |
| 1.5 | Perceptron outputs in plain language. . . . . | 18 |
| 2.1 | HMM functions, inputs, and outputs. . . . .   | 21 |
| 2.2 | HMM input data and model controls. . . . .    | 21 |

---

## Assignment 1

---

# Perceptron

---

## Problem: Student Exam Pass/Fail Prediction

### 1.1 Problem Description

The first website turns a supervised binary-classification lesson into a visible classroom map. Each synthetic student has two inputs: study hours per week,  $p_1 \in [0, 10]$ , and normalized attendance,  $p_2 \in [0, 10]$ . An attendance value of 7.5 represents 75%. The target  $t$  is 1 for Pass or 0 for Fail. The goal is to learn one straight line that places the green Pass examples on one side and the red Fail examples on the other.

This problem is suitable for a single neuron only when the two classes are linearly separable. The XOR preset deliberately violates that condition. It shows the model's limitation: no amount of repeating the same straight-line rule can separate labels placed at opposite corners. The design and update rule follow Chapter 4 of Hagan et al. [1], while the Perceptron originates in Rosenblatt's early work on trainable threshold systems [2].

**Safety boundary:** The points are invented and the result is not a grading, admissions, or student-risk decision. A real educational model would require representative data, fairness analysis, uncertainty measurement, governance, and human review.

### 1.2 Algorithm Description

#### 1.2.1 The chalk-line explanation

Imagine placing green and red cards on a floor. A robot has one piece of chalk. It guesses which side each card belongs on. When it is wrong, it turns or moves the chalk line; when it is right, it does nothing. One full visit to every card is an *epoch*. The learned line is the decision boundary.

The input vector, parameter vector, net input, and hard-limit output are

$$p = \begin{bmatrix} p_1 \\ p_2 \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad n = w^T p + b, \quad (1.1)$$

$$a = \text{hardlim}(n) = \begin{cases} 1, & n \geq 0, \\ 0, & n < 0. \end{cases} \quad (1.2)$$

The error is  $e = t - a$ . For learning rate  $\alpha$ , one mistake changes the parameters using

$$w_{\text{new}} = w_{\text{old}} + \alpha e p, \quad b_{\text{new}} = b_{\text{old}} + \alpha e. \quad (1.3)$$

The project fixes  $\alpha = 1$ , matching the unified rule in Hagan et al. [1]. If  $e = 1$ , the sample vector is added; if  $e = -1$ , it is subtracted; if  $e = 0$ , no parameter changes.

### 1.2.2 A numerical step

For  $p = [2, 3]^T$ ,  $t = 0$ ,  $w = [0, 0]^T$ , and  $b = 0$ , the net input is zero and `hardlim(0)` returns 1. Therefore  $e = 0 - 1 = -1$ , so the next parameters are  $w = [-2, -3]^T$  and  $b = -1$ . The website stores both old and new values so the animation can explain this exact correction.

### 1.2.3 Core code and data flow

```

1 net = float(self.weights @ sample + self.bias)
2 activation = int(self.hardlim(net))
3 error = int(t - activation)
4
5 if error != 0:
6     self.weights = self.weights + self.learning_rate * error * sample
7     self.bias = self.bias + self.learning_rate * error

```

Listing 1.1: One Perceptron update from `perceptron.py`.

Table 1.1: Perceptron implementation map.

| Function                                          | Input / return                                  | Role and calling flow                                                                       |
|---------------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>Perceptron(input_dim, learning_rate)</code> | 2 and 1.0; model object                         | Validates parameters and creates zero weights and bias. Called by the Flask training route. |
| <code>hardlim(net)</code>                         | Scalar or array; 0/1                            | Applies $n \geq 0$ . Called by <code>predict</code> and <code>train_step</code> .           |
| <code>_coerce_inputs(p)</code>                    | Vector or matrix; array plus single/-batch flag | Checks shape and finite values before matrix multiplication.                                |
| <code>_coerce_dataset(P,T)</code>                 | Sample matrix and targets; aligned arrays       | Checks row counts, non-empty data, and binary targets.                                      |
| <code>net_input(p)</code>                         | One or many points; score(s)                    | Computes <code>inputs @ weights + bias</code> ; NumPy broadcasts $b$ .                      |
| <code>predict(p)</code>                           | One or many points; class(es)                   | Calls <code>net_input</code> , then <code>hardlim</code> .                                  |
| <code>train_step(p,t)</code>                      | One point and target; diagnostics dictionary    | Predicts, calculates $e$ , updates immediately, and returns JSON-safe old/new state.        |
| <code>train_epoch(P,T)</code>                     | Complete set; step history                      | Calls <code>train_step</code> in submitted order once per row.                              |
| <code>fit(P,T,max_epochs)</code>                  | Data and limit; histories and final model       | Repeats epochs until accuracy is 1.0 or the limit is reached.                               |
| <code>/api/train</code>                           | JSON points; JSON training result               | Validates the request, trains a fresh model, and returns every animation snapshot.          |

The main data structures are a NumPy matrix  $P$  of shape  $(Q, 2)$ , a target vector  $T$  of length  $Q$ , a two-element weight array, a scalar bias, a list of step dictionaries, and a list of epoch summaries. Immediate updates make this online learning: sample  $q + 1$  sees the parameters produced by sample  $q$ .

### 1.3 Dataset Description

There is no personal student database. The user makes a temporary dataset by clicking the graph or loading one of three literal JavaScript presets. This is a synthetic design choice based on the supervised input/target-pair notation in Hagan et al. [1].

Table 1.2: Perceptron dataset fields.

| Field  | Allowed values | Meaning                                                                                       |
|--------|----------------|-----------------------------------------------------------------------------------------------|
| p1     | 0 to 10        | Synthetic study hours per week; horizontal graph coordinate.                                  |
| p2     | 0 to 10        | Normalized attendance; multiply by 10 for percent.                                            |
| target | 0 or 1         | Requested Fail or Pass class. This is the teaching answer, not a measured prediction outcome. |

The Clearly Separable preset contains ten points, five per class. The Boundary Edge preset also contains ten points but places several close to a possible line. The XOR-like preset contains four opposite-corner points. The server accepts 2 to 50 points, requires both labels, rejects non-finite or out-of-range values, and rejects identical coordinates carrying conflicting targets. Data exists only in browser memory and the request; no database is written.

## 1.4 Training

### 1.4.1 How to run and train

```

1 cd C:\Users\User\machine_learning\perceptron\student_perceptron_app
2 python -m venv .venv
3 .\.venv\Scripts\Activate.ps1
4 python -m pip install -r requirements.txt
5 python app.py

```

Listing 1.2: Run the Perceptron website.

Open <http://localhost:5000>, add both green and red points, and press **Train (Animated)**. The browser sends `{points:[...], max_epochs:100}` to `POST /api/train`. Flask creates a fresh `Perceptron(2,1.0)`, calls `fit`, saves the final local snapshot, and returns initial state, sample-by-sample history, epoch history, convergence status, boundary metadata, and final accuracy.

For direct Python use:

```

1 from perceptron import Perceptron
2
3 P = [[1, 2], [2, 1], [8, 8], [9, 7]]
4 T = [0, 0, 1, 1]
5 result = Perceptron(input_dim=2, learning_rate=1.0).fit(
6     P, T, max_epochs=100
7 )
8 print(result["final"])

```

Listing 1.3: Train without the website.

### 1.4.2 Parameters a learner may change

Table 1.3: Perceptron tuning guide.

| Parameter     | Current         | Effect of changing it                                                                                                                                                                                                           |
|---------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| learning_rate | 1.0             | Smaller values move the line less per mistake; larger values move it more. For a classic separable Perceptron, changing the positive step size usually changes the route and parameter scale, not the existence of a separator. |
| max_epochs    | 100             | A larger limit permits more passes but cannot make one line solve XOR. A smaller value returns sooner and may stop before a difficult separable set converges.                                                                  |
| Point order   | insertion order | Online updates depend on order, so the animation and final separating line can differ even when accuracy is the same.                                                                                                           |
| input_dim     | 2               | More features require matching input columns and lose the simple two-dimensional drawing.                                                                                                                                       |

The verified suite ran 12 Perceptron/API tests successfully, including exact hard-limit boundary behavior, the update equation, separable convergence, and bounded XOR non-convergence.

## 1.5 Website Design and Answer Flow

The browser owns the points, animation cursor, drawing, and telemetry. Flask owns request validation and the NumPy training operation. Canvas pixel coordinates use a downward  $y$  axis, while feature coordinates use an upward  $p_2$  axis. The conversion is

$$x = x_L + \frac{p_1}{10}W, \quad y = y_B - \frac{p_2}{10}H, \quad (1.4)$$

and the inverse subtracts the canvas margins, divides by the plot size, and multiplies by 10.

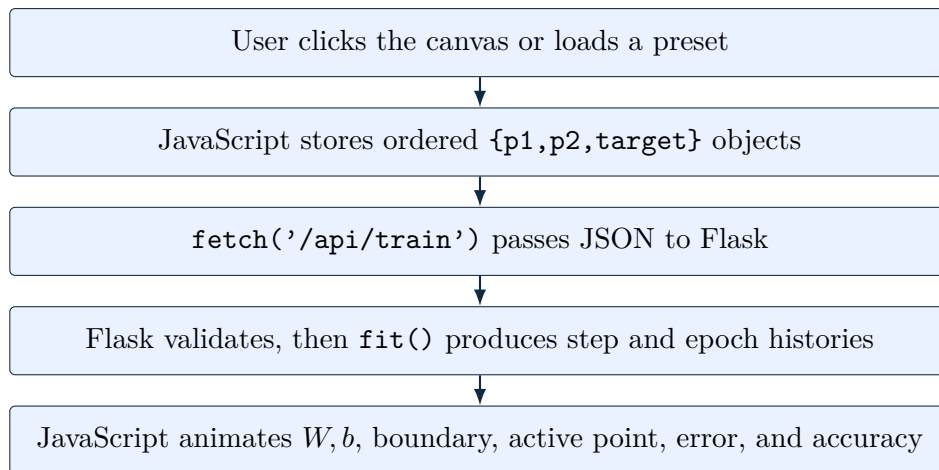


Figure 1.1: Perceptron website request and response flow.

The decision boundary is  $w_1p_1 + w_2p_2 + b = 0$ . The client safely draws a vertical line when  $w_2$  is approximately zero, a slope-intercept line otherwise, and no line when both weights are zero. The lightly shaded side satisfies  $w^T p + b \geq 0$ .

## 1.6 Comprehensive User Manual

### 1.6.1 Screen guide

Figures 1.2–1.5 form one complete recorded experiment. The large left card is the classroom map. The right column contains dataset controls, training buttons, speed, live mathematics, and the model's explanation.

## 1.6.2 Recorded website experiment: a learnable class boundary

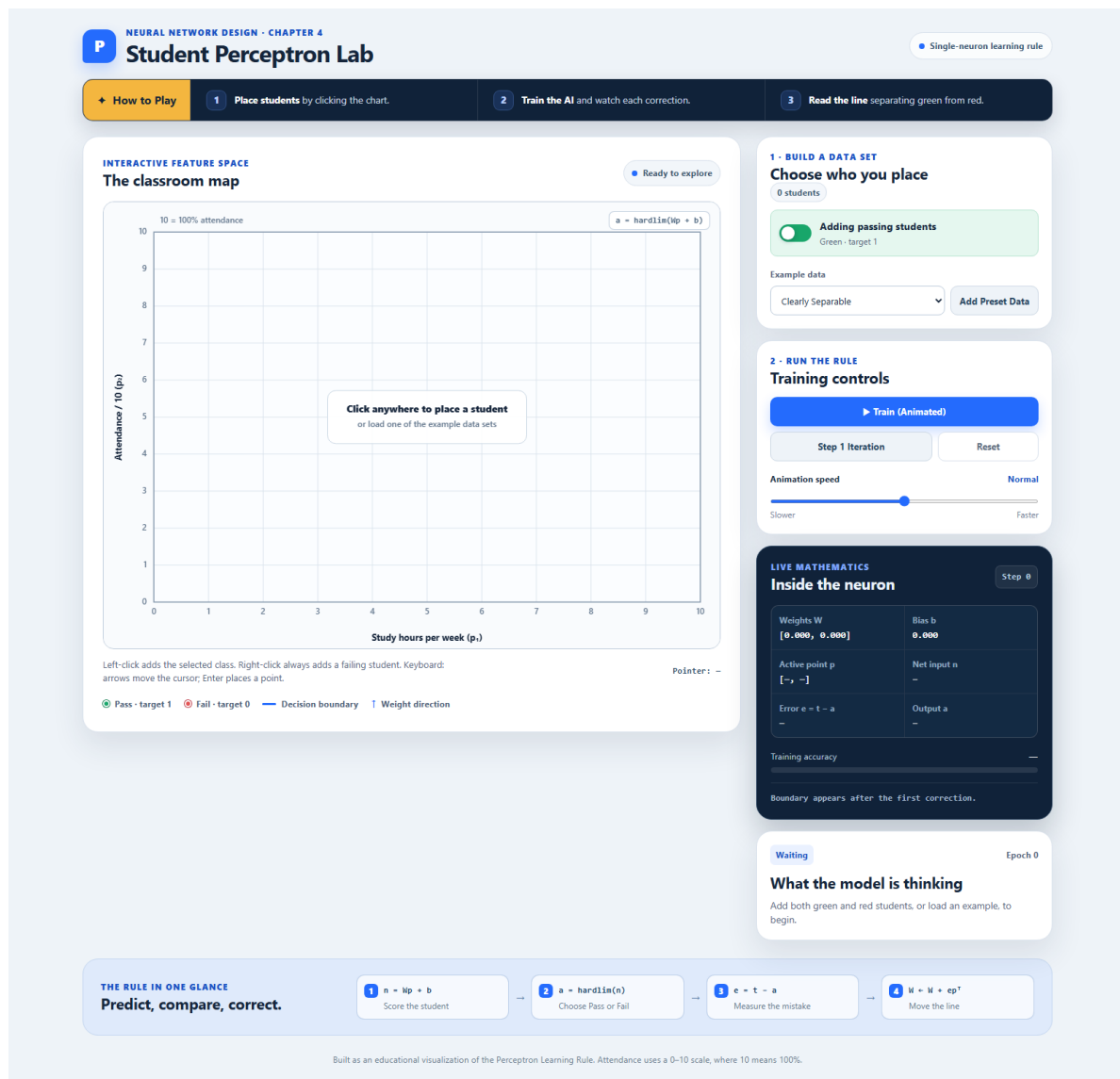


Figure 1.2: Stage 1 - initial Perceptron page. The classroom map is empty, the student counter is zero,  $W = [0, 0]$ ,  $b = 0$ , and no decision boundary exists yet.

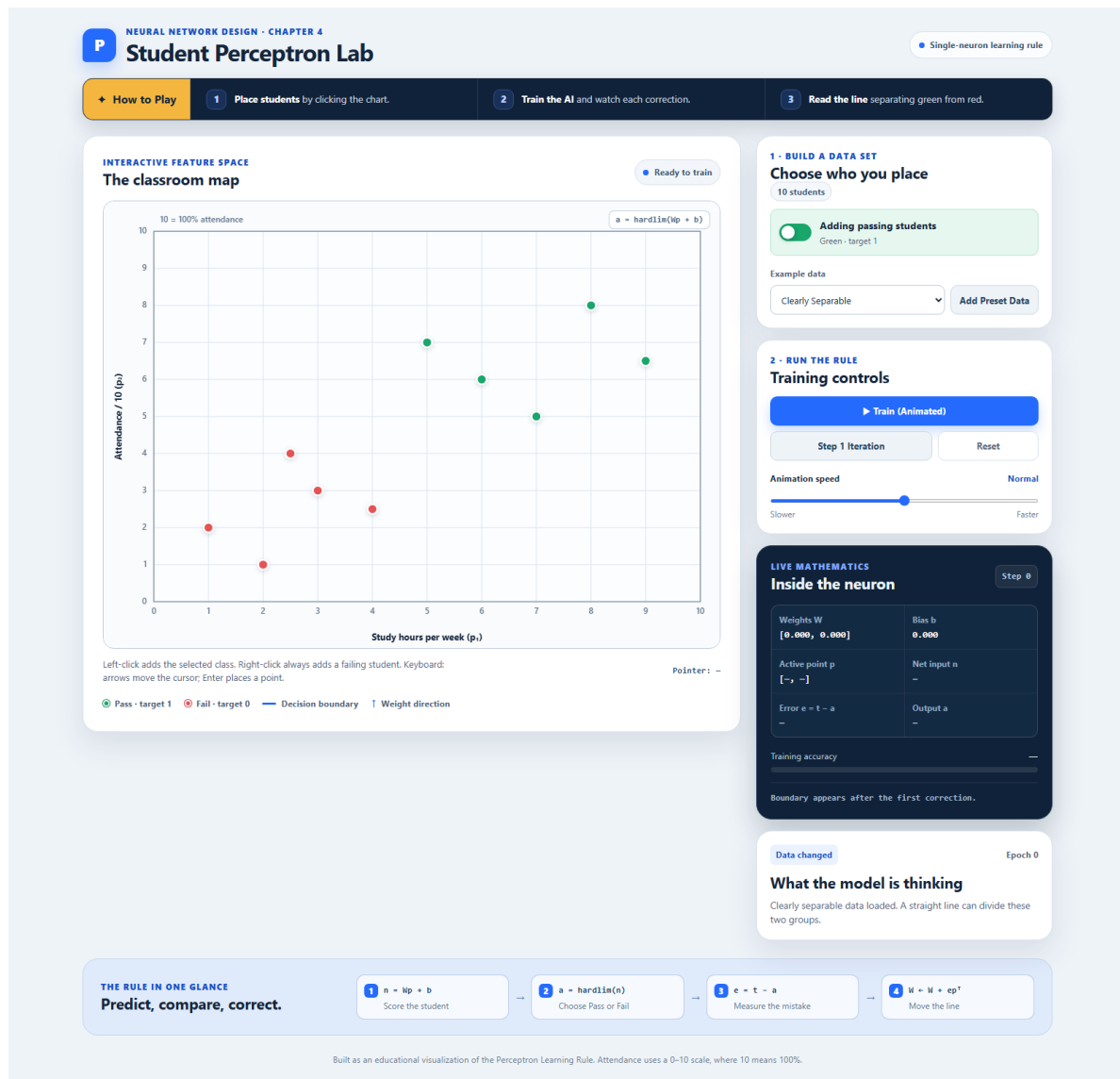


Figure 1.3: Stage 2 - input prepared. The Clearly Separable preset adds ten labeled students: five red Fail examples and five green Pass examples. Training has not started, so the displayed weights and bias remain zero.

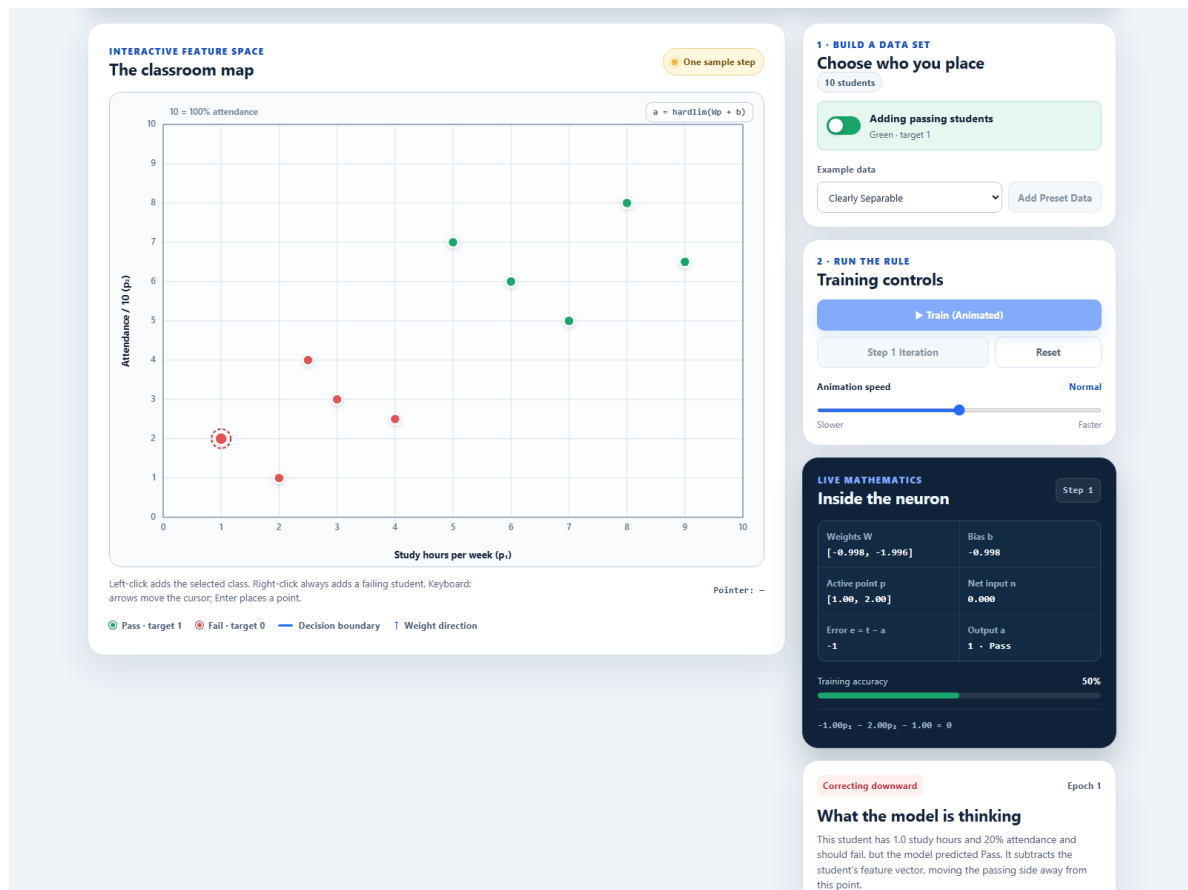


Figure 1.4: Stage 3 - one sample visit. The dashed ring identifies the active training point, while the telemetry reports its target, prediction, error, updated weights, bias, accuracy, epoch, and step.

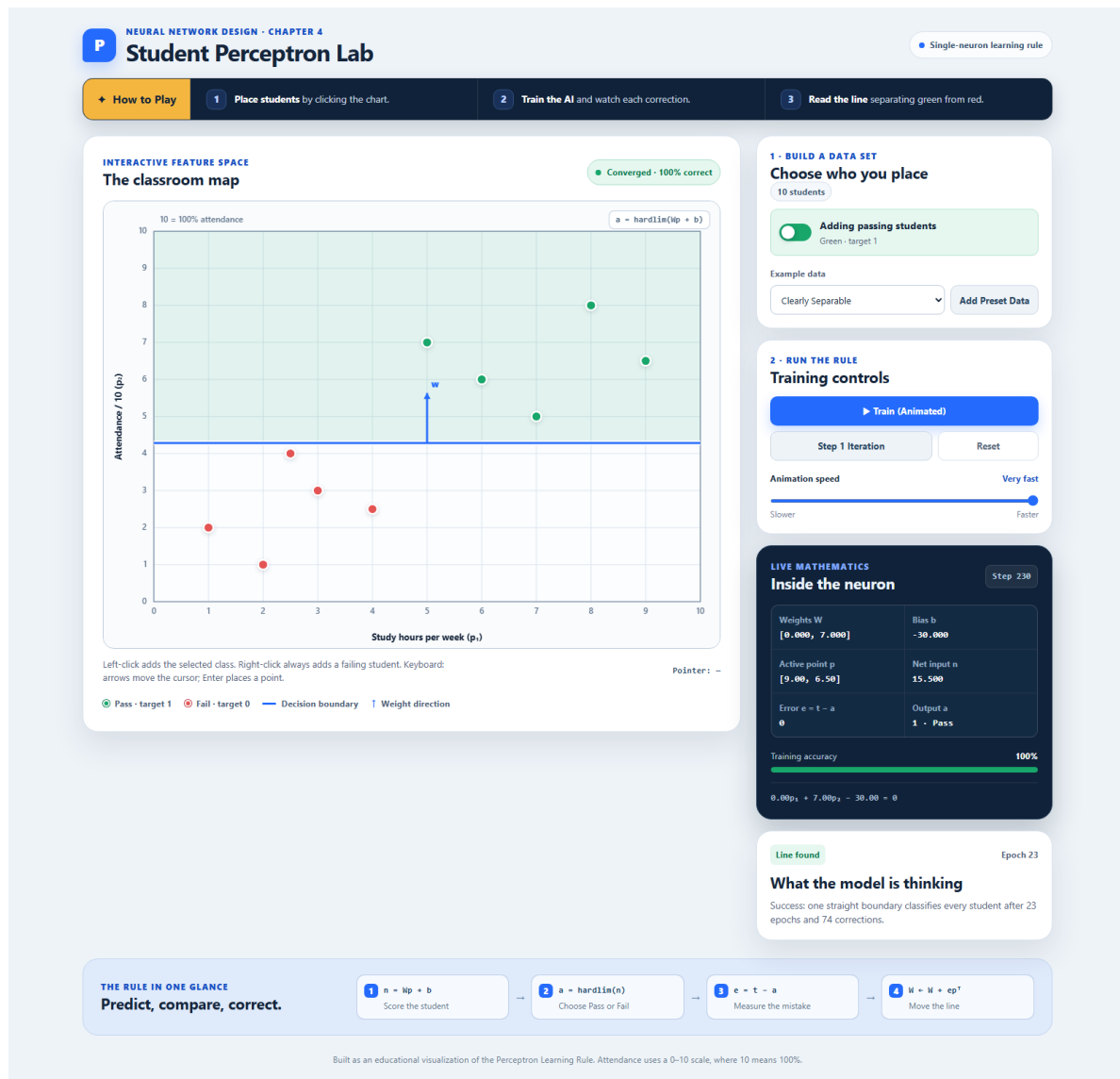


Figure 1.5: Stage 4 - final output. The page reports “Converged - 100% correct” after epoch 23 and step 230, with  $W = [0, 7]$ ,  $b = -30$ , and the blue boundary  $7p_2 - 30 = 0$ .

The recorded input is the ten-point Clearly Separable preset. The output means that this particular training set is separated perfectly by the horizontal line  $p_2 = 30/7 \approx 4.29$ . It does not claim perfect performance for unknown students. Figure 1.4 explains how a single mistake changes the parameters; Fig. 1.5 shows the accumulated result after all required passes.

### 1.6.3 Every input and control

#### Control

#### What the user does and what it means

Class switch

Green means new clicks receive target 1 (Pass); red means target 0 (Fail). A right-click always adds Fail.

|                    |                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classroom map      | Horizontal position is study hours; vertical position is attendance divided by 10. Click inside the numbered square only.                                 |
| Example data list  | Choose Clearly Separable, Boundary Edge Case, or XOR-like, then press <b>Add Preset Data</b> . Presets are added to any existing points if space remains. |
| Train (Animated)   | Requests the complete training history, resets to the zero model, and plays every stored sample visit.                                                    |
| Step 1 Iteration   | Requests the same history if needed, then advances exactly one sample.                                                                                    |
| Animation speed    | Changes playback speed only; it does not change learning mathematics.                                                                                     |
| Reset              | Cancels animation, deletes temporary points, restores zero parameters, and returns placement to Pass.                                                     |
| Keyboard operation | Focus the graph, use arrow keys to move the cursor, and press Enter or Space to place a point.                                                            |

---

#### 1.6.4 How to perform a successful experiment

1. Choose **Clearly Separable** and press **Add Preset Data**.
2. Confirm that the counter shows ten students and both red and green points are visible.
3. Press **Step 1 Iteration** once. Read the active point  $p$ , target  $t$ , old score  $n$ , output  $a$ , error  $e = t - a$ , weights  $W$ , and bias  $b$ .
4. Continue stepping or press **Train (Animated)**. A dashed ring marks the sample being visited. The line and blue weight arrow move only after a mistake.
5. Stop when the badge says **Converged - 100% correct**. Read the number of epochs and corrections in the narration.
6. Repeat with XOR-like data. The expected result is **No single line fits**; this is a model-capacity lesson, not a server failure.

### 1.6.5 How to read every output

Table 1.5: Perceptron outputs in plain language.

| Output           | Interpretation                                                                                                     |
|------------------|--------------------------------------------------------------------------------------------------------------------|
| $W = [w_1, w_2]$ | Two steering numbers. Their direction is perpendicular to the line and points toward the predicted Pass side.      |
| $b$              | The sliding control that moves the line away from the origin.                                                      |
| $p$              | The current student's two feature coordinates.                                                                     |
| $n$              | Raw score before thresholding; it is not a probability.                                                            |
| $a$              | Current class, 1 for Pass if $n \geq 0$ , otherwise 0.                                                             |
| $e = t - a$      | 0 means correct; 1 or -1 means a correction was required.                                                          |
| Accuracy         | Share of the currently placed examples classified correctly. It is training accuracy, not future-student accuracy. |
| Boundary         | Blue line where $n = 0$ ; green shade is the predicted Pass half-plane.                                            |
| Epoch / step     | Epoch counts full dataset passes; step counts individual sample visits.                                            |

### 1.6.6 Troubleshooting

If training asks for both classes, add at least one green and one red point. If the line never becomes perfect, test whether a single straight line could separate the colors by eye. If no point appears, click inside the plot. If all initial outputs are Pass, remember that zero weights give  $n = 0$  and this implementation assigns the boundary to class 1. If data is edited, the old line is intentionally cleared because it no longer describes the current points.

## 1.7 Conclusion

The Perceptron chapter demonstrates the smallest complete supervised learning loop: predict, compare, correct, repeat. Its greatest teaching strength is that the parameters, line, error, and convergence limit are visible. Its greatest limitation is equally visible: one hard-limit neuron can draw only one linear boundary.

---

## Assignment 2

---

# Hidden Markov Model

---

## Problem: Hidden Coin Detective

### 2.1 Problem Description

Imagine that a person secretly chooses between a fair coin and a heads-biased coin. The coin itself is hidden, but the observer records each Head (H) or Tail (T). The task is to infer the most likely hidden coin used at every toss and to show how uncertainty changes through time. This is a sequence problem: the order of observations matters.

### 2.2 The Algorithm

#### 2.2.1 Model pieces

A hidden Markov model (HMM) contains hidden states, visible symbols, and three probability structures [3]:

- $\pi$ : starting probabilities for Fair and Biased;
- $A$ : transition probabilities, such as Fair staying Fair;
- $B$ : emission probabilities, such as Biased producing H.

With default controls the app constructs

$$\pi = (0.5, 0.5), \quad A = \begin{bmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5 & 0.5 \\ 0.85 & 0.15 \end{bmatrix}, \quad (2.1)$$

where emission columns are H and T. A Markov assumption says the next hidden coin depends on the current coin, not the full earlier history. An emission depends on the current hidden coin.

### 2.2.2 Viterbi: the single best hidden route

Multiplying many probabilities can underflow toward zero, so the implementation adds logarithms. Let  $\delta_t(j)$  be the best log-score for all routes ending in state  $j$  at time  $t$ . Then

$$\delta_1(j) = \log \pi_j + \log b_j(o_1), \quad (2.2)$$

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log a_{ij}] + \log b_j(o_t). \quad (2.3)$$

For every maximum, a backpointer  $\psi_t(j)$  stores which earlier state won. After the final toss, following those pointers backward reconstructs the best path. With  $T$  observations and  $N$  states, work grows as  $O(TN^2)$ .

```

1 for time in range(1, len(observations)):
2     for current in range(state_count):
3         choices = [score[time - 1][previous]
4                     + log(transition[previous][current])
5                     for previous in range(state_count)]
6         winner = max(range(state_count), key=lambda p: choices[p])
7         score[time][current] = (choices[winner]
8                                 + log(emission[current][symbol]))
9         backpointer[time][current] = winner
10 # Start at the best final state and follow backpointers toward time zero.
```

Listing 2.1: Viterbi recursion, simplified from HMM/dist/hmm.py.

### 2.2.3 Forward-backward: belief at each position

The best complete route is not the same as the probability of each state at one position. Forward values summarize all ways to reach a state; backward values summarize all possible futures. Their product is normalized to obtain  $P(q_t = j \mid o_1, \dots, o_T)$ . The website therefore reports both the Viterbi route and a posterior Fair/Biased percentage at every toss.

Table 2.1: HMM functions, inputs, and outputs.

| Function                       | Receives                           | Produces                                           |
|--------------------------------|------------------------------------|----------------------------------------------------|
| <code>cleanObservations</code> | Text tokens                        | Validated H/T array of length 1–60                 |
| <code>checkProbability</code>  | Name and numeric value             | Safe probability or a clear error                  |
| <code>makeModel</code>         | Heads chances and stay chances     | $\pi$ , $A$ , and $B$ arrays whose rows sum to one |
| <code>viterbi</code>           | Observation array and model        | Best state-index path and path probability         |
| <code>forwardBackward</code>   | Same sequence and model            | Posterior pair for every position                  |
| <code>analyse</code>           | Raw observations and four settings | Named path, summaries, posteriors, and model       |
| <code>renderResult</code>      | Analysis object                    | Route cards, timeline, chart, and explanation      |

JavaScript performs the live calculation; `HMM/dist/hmm.py` is a matching Python teaching implementation used by the tests. Both pass arrays and plain objects explicitly, which makes every probability inspectable.

## 2.3 Dataset Description

This lab has no stored training table. Its “data” is one observation sequence entered by the user plus four probability settings. Spaces, commas, or a compact string such as `HHHHTHTT` are accepted and converted to one H/T array.

Table 2.2: HMM input data and model controls.

| Item                | Allowed/default     | Plain-language meaning                                    |
|---------------------|---------------------|-----------------------------------------------------------|
| Observation         | H or T, 1–60 tosses | What the observer can actually see, in order.             |
| Fair heads chance   | 0.50 default        | Chance that Fair emits H; its T chance is one minus this. |
| Biased heads chance | 0.85 default        | Chance that Biased emits H.                               |
| Fair stay chance    | 0.80 default        | Chance the hidden source remains Fair for the next toss.  |
| Biased stay chance  | 0.80 default        | Chance it remains Biased for the next toss.               |
| Initial state       | 0.50 each           | Before seeing a toss, both coins are equally likely.      |

The supplied presets (heads-heavy, tails-heavy, and alternating) are synthetic sequences

chosen to reveal behavior. They were not collected from experiments.

## 2.4 Training and Parameter Experimentation

This website does *not* train probabilities and does not implement the Baum-Welch algorithm. It performs decoding and inference using probabilities the user supplies. This distinction prevents calling every model calculation “training.” To run it:

```
1 cd HMM
2 python -m http.server 4173 --directory dist
3 # Then open http://localhost:4173
4
5 # Optional Python calculation from inside HMM/dist:
6 python -c "from hmm import analyse_json; print(analyse_json('H H T'))"
```

Changing a heads probability changes what each state tends to emit. For example, raising Biased heads chance makes a long H run stronger evidence for Biased. Raising a stay probability penalizes frequent switching and produces longer same-state segments. Lowering stay probabilities makes alternating states easier. A probability of exactly zero would make a path impossible; the UI keeps sliders between 5% and 95% to avoid extreme numerical and teaching cases.

## 2.5 Website

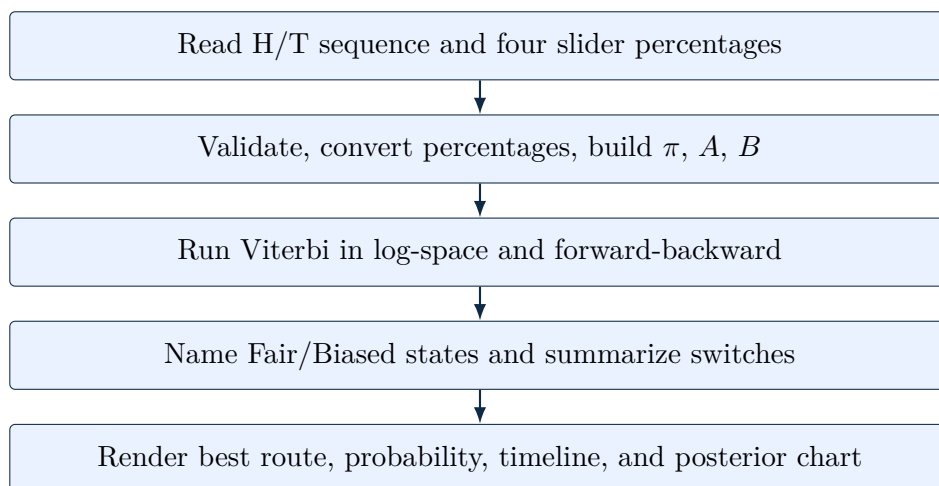


Figure 2.1: Hidden Coin Detective inference flow.

## 2.6 Comprehensive User Manual

## 2.6.1 Recorded website experiment: seven heads with a strongly biased coin

Figures 2.2–2.5 separate model settings, Viterbi’s complete route, and forward-backward belief at one selected time.

**Behind the Curtain**  
Hidden Markov Model lab

Offline model ready

**RABINER - PROBLEM 2 - DECODING**

# Which coin was hiding behind each toss?

You hear the outcomes, but never see the coin. Change the evidence or the dealer's habits, then let Viterbi reveal the most likely hidden route.

**FIRST TIME HERE? NO MATHS NEEDED.**  
**Reveal the hidden coin in three moves**

The dealer is behind a curtain. You hear each toss, but you cannot see which coin made it. Your job is to ask the model for its best explanation.

- Enter what you heard**  
Use H for heads and T for tails.
- Keep or change the setup**  
The starting values already make a complete example.
- Decode, then explore**  
Select any toss to understand that step.

Or try a ready-made sequence: Original example Heads streak Tails streak Alternating

---

**01 Set the experiment** Reset

**Observed tosses** 9 tosses  
This is what the dealer announces—not the coin you are trying to guess. Type H or T, spaces, commas, and dashes are okay.

HHHHHTTT Decode

Allowed: 1-60 tosses using only H and T, spaces, commas, and dashes are okay.

**How the coins behave**  
A higher heads probability means that coin is more likely to produce H. The remaining probability belongs to tails.

**Fair coin** Heads probability: 50%  
Allowed: 5-95%. A fair coin usually starts at 50%.

**Biased coin** Heads probability: 85%  
Allowed: 5-95%. Higher means heads appears more often.

**Dealer's switching habits** Advanced

---

**02 The hidden route** Decoded 9 tosses

**PATH PROBABILITY**  
**0.0582%**  
Exact route = all tosses; not a confidence score

**MOST LIKELY EXPLANATION**  
**Biased × 5 → Fair × 4**

**Plain meaning:** this is the most likely complete coin route under the settings on the left. It is a model prediction, not proof of what the hidden dealer actually did.

HEARD: H H H H H T T T  
HIDDEN: BIASED BIASED BIASED BIASED BIASED FAIR FAIR FAIR FAIR

**STEP 1 - HEARD H**  
**Viterbi chooses the biased coin**  
At the first toss, Viterbi combines the 50.0% starting chance with the 85.0% chance that the biased coin emits this result. That gives the stronger opening route.

The two bars answer "which coin is likely at this one step?" using every observed toss. The route above answers a different question: "which single start-to-finish path is best?" They can occasionally disagree at one step.

Far: 25.2%  
Biased: 74.8%  
Posterior probability using all 9 observations

---

**03 How Viterbi thinks**

- Start**  
Score each possible starting coin.
- Compare**  
Keep only the best route into each state.
- Backtrack**  
Follow the winning pointers to the start.

---

**04 PLAIN LANGUAGE GUIDE**  
**What does everything on the screen mean?**

The app is a safe place to experiment. Change one setting, decode again, and notice how the hidden route responds. There is no score to win.

| WHAT YOU CONTROL<br>Inputs: evidence and assumptions |                                                                                                                                     | WHAT THE MODEL RETURNS<br>Outputs: the model's explanation |                                                                                                                                           |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Observed tosses</b>                               | The H and T results you heard. These are the visible evidence.                                                                      | <b>Hidden route</b>                                        | Viterbi's single best Fair/Biased path for the complete sequence.                                                                         |
| <b>Heads probability</b>                             | How often each coin is expected to land heads. Tails receives whatever probability remains. For example, 85% heads means 15% tails. | <b>Path probability</b>                                    | The probability of that exact hidden route together with all the observed tosses. Small values are normal, especially for long sequences. |
| <b>Keep the coin</b>                                 | How likely the dealer is to reuse the same coin on the next toss. A high value encourages longer runs with fewer switches.          | <b>Fair / Biased bars</b>                                  | The probability of each coin at the selected step after considering the whole sequence. The two bars always total 100%.                   |

**The rule to remember**  
HMM results depend on the assumptions you enter. If you change how biased the coins are or how often the dealer switches, the best route may change. That sensitivity is part of the lesson—not a mistake in the app.

**Things to notice while you play**

- A run of heads usually supports the biased coin.
- Higher "keep" values make the model resist frequent switching.
- Tails usually supports the fair coin under the starting values.
- Similar coins make the tosses less informative.

Based on the hidden two-coin experiment in L. R. Rabiner's HMM tutorial. Algorithm local JavaScript. Python teaching version included.

Figure 2.2: Stage 1 - initial HMM page. The default nine-toss example is already decoded, showing the complete interface: observations, probability controls, Viterbi route, timeline, posterior bars, and plain-language guide.

Behind the Curtain  
Hidden Markov Model lab
Offline model ready

RABINER - PROBLEM 2 - DECODING

# Which coin was hiding behind each toss?

You hear the outcomes, but never see the coin. Change the evidence or the dealer's habits, then let Viterbi reveal the most likely hidden route.

**FIRST TIME HERE? NO MATHS NEEDED.**

## Reveal the hidden coin in three moves

The dealer is behind a curtain. You hear each toss, but you cannot see which coin made it. Your job is to ask the model for its best explanation.

- Enter what you heard**  
Use H for heads and T for tails.
- Keep or change the setup**  
The starting values already make a complete example.
- Decode, then explore**  
Select any toss to understand that step.

Or try a ready-made sequence    Original example    Heads streak    Tails streak    Alternating

### 01 Set the experiment

Reset

**Observed tosses**  
This is what the dealer announces—not the coin you are trying to guess. Type H or T, spaces and dashes are optional.  
H H H H H H    **Decode**

Allowed: 1-60 tosses using only H and T, spaces, commas, and dashes are okay.

**How the coins behave**  
A higher heads probability means that coin is more likely to produce H. The remaining probability belongs to tails.

**Fair coin**  
Heads probability: 50%  
Allowed: 5-95%. A fair coin usually starts at 50%.

**Biased coin**  
Heads probability: 90%  
Allowed: 5-95%. Higher means heads appears more often.

**Dealer's switching habits**  
Advanced

### 02 The hidden route

Decoded 9 tosses

**MOST LIKELY EXPLANATION**  
Biased × 5 → Fair × 4  
PATH PROBABILITY: 0.0582%  
Exact route = all tosses; not a confidence score

**Plain meaning:** this is the most likely complete coin route under the settings on the left. It is a model prediction, not proof of what the hidden dealer actually did.

HEARD: H H H H H T H T T  
HIDDEN: BIASED BIASED BIASED BIASED BIASED FAIR FAIR FAIR FAIR

**STEP 1 - HEARD H**  
Viterbi chooses the biased coin  
At the first toss, Viterbi combines the 50.0% starting chance with the 85.0% chance that the biased coin emits this result. That gives the stronger opening route.  
Fair: 25.2%  
Biased: 74.8%  
Posterior probability using all 9 observations

The two bars answer "which coin is likely at this one step?" using every observed toss. The route above answers a different question: "which single start-to-finish path is best?" They can occasionally disagree at one step.

### 03 How Viterbi thinks

One careful pass forward, then one clean trace backward.

- Start**  
Score each possible starting coin.
- Compare**  
Keep only the best route into each state.
- Backtrack**  
Follow the winning pointers to the start.

### 04 - PLAIN LANGUAGE GUIDE

#### What does everything on the screen mean?

The app is a safe place to experiment. Change one setting, decode again, and notice how the hidden route responds. There is no score to win.

| WHAT YOU CONTROL                 |                                                                                                                                     | WHAT THE MODEL RETURNS           |                                                                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Inputs: evidence and assumptions |                                                                                                                                     | Outputs: the model's explanation |                                                                                                                                           |
| <b>Observed tosses</b>           | The H and T results you heard. These are the visible evidence.                                                                      | <b>Hidden route</b>              | Viterbi's single best Fair/Biased path for the complete sequence.                                                                         |
| <b>Heads probability</b>         | How often each coin is expected to land heads. Tails receives whatever probability remains. For example, 85% heads means 15% tails. | <b>Path probability</b>          | The probability of that exact hidden route together with all the observed tosses. Small values are normal, especially for long sequences. |
| <b>Keep the coin</b>             | How likely the dealer is to reuse the same coin on the next toss. A high value encourages longer runs with fewer switches.          | <b>Fair / Biased bars</b>        | The probability of each coin at the selected step after considering the whole sequence. The two bars always total 100%.                   |

**The rule to remember**  
HMM results depend on the assumptions you enter. If you change how biased the coins are or how often the dealer switches, the best route may change. That sensitivity is part of the lesson—not a mistake in the app.

**Things to notice while you play**

- A run of heads usually supports the biased coin.
- Higher "keep" values make the model resist frequent switching.
- Tails usually supports the fair coin under the starting values.
- Similar coins make the tosses less informative.

Based on the hidden two-coin experiment in L. R. Rabiner's HMM tutorial.    Algorithms local JavaScript. Python teaching version included.

Figure 2.3: Stage 2 - input prepared but not decoded. The observation field contains seven Heads; Biased-heads is 90%, Fair-stay is 70%, and Biased-stay is 85%. The old “Decoded 9 tosses” result remains visible until Decode is pressed.

Behind the Curtain  
Hidden Markov Model lab
Offline model ready

RABINER - PROBLEM 2 - DECODING

# Which coin was hiding behind each toss?

You hear the outcomes, but never see the coin. Change the evidence or the dealer's habits, then let Viterbi reveal the most likely hidden route.

**FIRST TIME HERE? NO MATHS NEEDED.**

## Reveal the hidden coin in three moves

The dealer is behind a curtain. You hear each toss, but you cannot see which coin made it. Your job is to ask the model for its best explanation.

- Enter what you heard**  
Use H for heads and T for tails.
- Keep or change the setup**  
The starting values already make a complete example.
- Decode, then explore**  
Select any toss to understand that step.

Or try a ready-made sequence    Original example    Heads streak    Tails streak    Alternating

### 01 Set the experiment

Reset

**Observed tosses**  
This is what the dealer announces—not the coin you are trying to guess. Type H or T, spaces and dashes are optional.  
H H H H H H H    **Decode**  
Allowed: 1-60 tosses using only H and T, spaces, commas, and dashes are okay.

**How the coins behave**  
A higher heads probability means that coin is more likely to produce H. The remaining probability belongs to tails.

**Fair coin**  
Heads probability: 50%  
Allowed: 5-95%. A fair coin usually starts at 50%.

**Biased coin**  
Heads probability: 90%  
Allowed: 5-95%. Higher means heads appears more often.

**Dealer's switching habits**  
Advanced

### 02 The hidden route

Decoded 7 tosses

**MOST LIKELY EXPLANATION**  
**Biased × 7**  
PATH PROBABILITY: **9.02%**  
Exact route + all tosses; not a confidence score.

**Plain meaning:** this is the most likely complete coin route under the settings on the left. It is a model prediction, not proof of what the hidden dealer actually did.

|        | t1     | t2     | t3     | t4     | t5     | t6     | t7     |
|--------|--------|--------|--------|--------|--------|--------|--------|
| HEARD  | H      | H      | H      | H      | H      | H      | H      |
| HIDDEN | BIASED | BIASED | BIASED | BIASED | BIASED | BIASED | BIASED |

**STEP 1 - HEARD H**  
**Viterbi chooses the biased coin**  
At the first toss, Viterbi combines the 50.0% starting chance with the 90.0% chance that the biased coin emits this result. That gives the stronger opening route.

The two bars answer "which coin is likely at this one step?" using every observed toss. The route above answers a different question: "which single start-to-finish path is best?" They can occasionally disagree at one step.

Fair: 24.7%  
Biased: 75.3%  
Posterior probability using all 7 observations

### 03 How Viterbi thinks

One careful pass forward, then one clean trace backward.

- Start**  
Score each possible starting coin.
- Compare**  
Keep only the best route into each state.
- Backtrack**  
Follow the winning pointers to the start.

### 04 - PLAIN LANGUAGE GUIDE

#### What does everything on the screen mean?

The app is a safe place to experiment. Change one setting, decode again, and notice how the hidden route responds. There is no score to win.

| WHAT YOU CONTROL                 |                                                                                                                                     | WHAT THE MODEL RETURNS           |                                                                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Inputs: evidence and assumptions |                                                                                                                                     | Outputs: the model's explanation |                                                                                                                                           |
| <b>Observed tosses</b>           | The H and T results you heard. These are the visible evidence.                                                                      | <b>Hidden route</b>              | Viterbi's single best Fair/Biased path for the complete sequence.                                                                         |
| <b>Heads probability</b>         | How often each coin is expected to land heads. Tails receives whatever probability remains. For example, 85% heads means 15% tails. | <b>Path probability</b>          | The probability of that exact hidden route together with all the observed tosses. Small values are normal, especially for long sequences. |
| <b>Keep the coin</b>             | How likely the dealer is to reuse the same coin on the next toss. A high value encourages longer runs with fewer switches.          | <b>Fair / Biased bars</b>        | The probability of each coin at the selected step after considering the whole sequence. The two bars always total 100%.                   |

**The rule to remember**  
HMM results depend on the assumptions you enter. If you change how biased the coins are or how often the dealer switches, the best route may change. That sensitivity is part of the lesson—not a mistake in the app.

**Things to notice while you play**

- A run of heads usually supports the biased coin.
- Higher "keep" values make the model resist frequent switching.
- Tails usually supports the fair coin under the starting values.
- Similar coins make the tosses less informative.

Based on the hidden two-coin experiment in L. R. Rabiner's HMM tutorial.    Algorithm local JavaScript. Python teaching version included.

Figure 2.4: Stage 3 - HMM output. For seven Heads under the stated probabilities, Viterbi returns Biased × 7 with an exact path-and-observations probability of 9.02%.

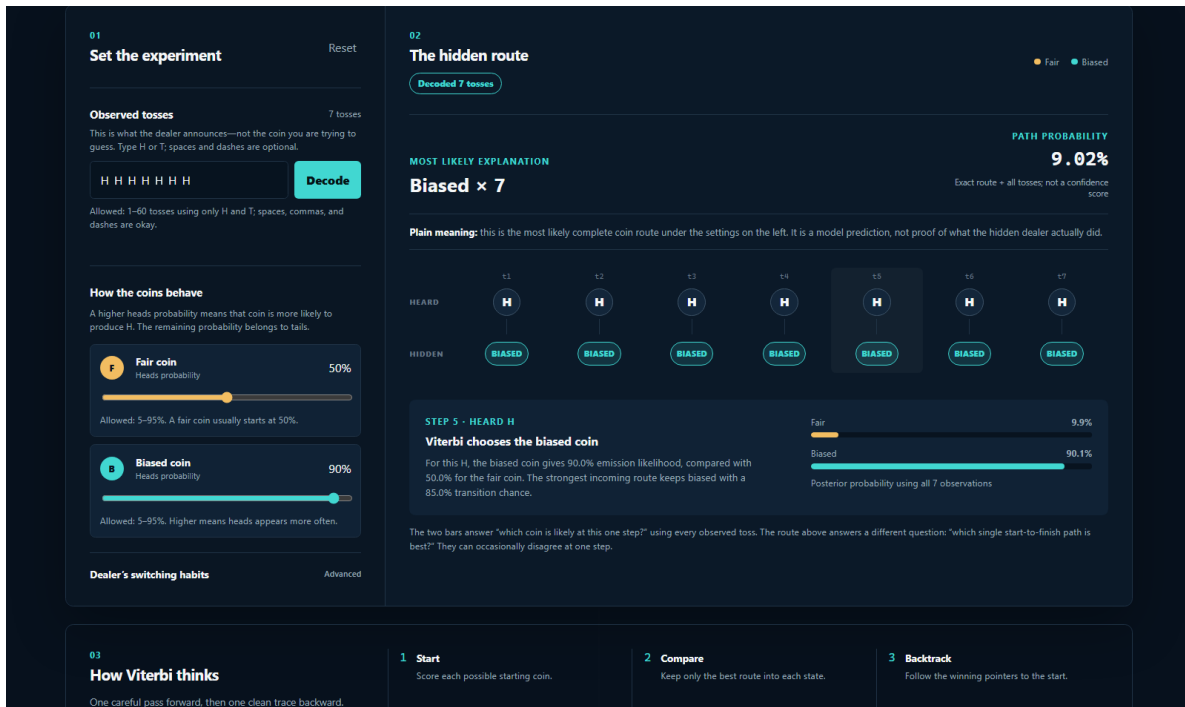


Figure 2.5: Stage 4 - additional posterior information. At toss 5, which is H, the forward-backward result assigns 9.9% to Fair and 90.1% to Biased. The two bars total 100%.

This experiment therefore records two related outputs: Viterbi's best complete story is Biased at all seven positions, while the position-specific posterior at step 5 is 90.1% Biased rather than absolute certainty. Path probability and posterior probability answer different questions.

1. Serve `HMM/dist` and open the page.
  2. Type H and T symbols. Separators may be spaces or commas, or none at all. Do not include other letters. Select a preset to learn by example.
  3. Move the four sliders only if exploring a different coin story. The number beside each slider is the exact percentage used.
  4. Press **Decode hidden coins**. Reset restores the original sequence and default model.
- **Most likely route** is the single complete state sequence with the greatest model probability, for example F–F–B–B. It is an inference, not observed truth.
  - **Path probability** is the joint probability of that route and all observed symbols. It can be small simply because long exact sequences are rare; it is not classifier confidence.
  - **Switch count** says how often adjacent inferred states differ.
  - **Timeline** aligns each visible H/T with its Viterbi state.
  - **Posterior chart** gives Fair and Biased percentages at each position after considering the entire observation sequence. These two bars sum to 100% and can disagree with how decisive the single route looks.

- **Model summary** restates the emissions/transitions actually used, making slider experiments reproducible.

In Fig. 2.4, a heads-only run favors the biased coin at every position. If decoding is disabled, add at least one valid symbol. If percentages behave unexpectedly, remember that “stay” describes the next hidden state, while “heads” describes the visible outcome from the current state.

## 2.7 Conclusion

This project shows why sequence models need both memory and uncertainty. Viterbi answers “Which complete hidden story is best?” while forward-backward answers “How plausible is each hidden state here?” Keeping both avoids a common misinterpretation.

---

## Overall Conclusion

---

The two assignment problems show learning and probabilistic reasoning from different directions. The Perceptron changes weights after mistakes until one line separates suitable labeled points. The HMM keeps its states hidden and uses emissions, transitions, Viterbi decoding, and posterior probabilities to explain an observed sequence. Together they emphasize that a model's structure must match the question: a static boundary and a time-dependent hidden story are not interchangeable.

Both websites expose the reasoning behind the output. Perceptron users see the line, weights, errors, and epochs; HMM users see the route, switches, path probability, and state posterior at each toss. These explanations make the algorithms testable while preserving the limits of synthetic educational data.

## Appendix A

# Verification and Reproduction

## A.1 Commands verified in this workspace

The following checks were executed against the submitted sources before this report was produced.

| Project    | Verification command                                                                                                | Observed result                      |
|------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| Perceptron | <code>python -m unittest discover -s tests -v</code>                                                                | 12 tests passed                      |
| HMM        | <code>python -m unittest discover -s tests -p test_hmm.py -v;</code><br><code>node tests/test_browser_hmm.js</code> | 5 Python tests; browser tests passed |

The Perceptron checks include exact hard-limit behavior, update equations, API validation, separable convergence, and bounded non-convergence. The HMM checks compare the Python and browser-facing calculations for validation, Viterbi, and forward-backward behavior.

## A.2 One-page run checklist

1. Install each project's declared dependencies before starting its server.
2. For the Perceptron, submit labeled points and watch whether the line converges; for HMM, enter only H/T observations and distinguish decoding from training.
3. Serve the HMM `dist` folder over localhost so its scripts load normally.
4. Wait for the page's ready indicator before entering a prediction.
5. Keep the units printed beside each field and interpret the evidence panel together with the main label.

---

## Appendix B

---

# Glossary for a First-Time Reader

---

| Word                 | Plain meaning in this report                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Algorithm            | A precise recipe of steps a computer follows.                                                                                                                |
| Feature              | An input fact, such as height, glucose, or one pixel value.                                                                                                  |
| Target/label         | The example answer supplied during supervised learning.                                                                                                      |
| Training             | Adjusting a model from examples; the Perceptron trains, while the current HMM page performs inference from user-set probabilities rather than training them. |
| Inference/prediction | Applying an existing model to a new input.                                                                                                                   |
| Parameter            | A learned or chosen number that affects behavior, such as a Perceptron weight or an HMM transition probability.                                              |
| Epoch                | One complete pass through all training examples.                                                                                                             |
| Accuracy             | Fraction of checked labels predicted correctly; its meaning depends completely on which rows were checked.                                                   |
| Overfitting          | Learning training details so narrowly that new-data performance becomes worse.                                                                               |
| Posterior            | Probability of a hidden state after considering the complete observation sequence.                                                                           |
| JSON                 | A text structure of names, arrays, numbers, and objects shared with the browser.                                                                             |
| API                  | A defined request/response doorway between pieces of software.                                                                                               |

---

## Bibliography

---

- [1] M. T. Hagan, H. B. Demuth, M. H. Beale, and O. De Jesús, *Neural Network Design*, 2nd ed. Stillwater, OK, USA, 2014. [Online]. Available: <https://hagan.okstate.edu/nnd.html>
- [2] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519.
- [3] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989, doi: 10.1109/5.18626.